Multiplayer Oyun Sistemlerinde Bilinmesi Gerekenler: Bomberman Projemiz Üzerinden Kapsamlı Bir İnceleme

Aşağıda, bu projede adım adım ne yaptığımızı, neden yaptığımızı ve bunların multiplayer sistemlerdeki genel karşılıklarını anlatıyorum. Kod hafızamda duruyor; üzerine ekleme yapmaya hazırım.

---

1. Temel Mimari Kararı: Neden Client-Server?

Ne yaptık: Tek bir cihazda çalışan bir oyun yerine, bir **sunucu** (otorite) ve birden fazla **istemci** (oyuncu) yazdık. Sunucu tüm oyun mantığını hesaplıyor, istemciler sadece görüntülüyor ve giriş gönderiyor.

Neden: Multiplayer'da üç temel mimari vardır:

- **Peer-to-peer (P2P):** Her istemci diğerleriyle doğrudan konuşur. Basit ama **hile (cheat) koruması zayıf** ve **tutarlılık (consistency)** zor.
- **Client-server (authoritative):** Sunucu tek doğruluk kaynağıdır (single source of truth). Oyuncu "ben buradayım" der, sunucu "hayır, duvardasın, orada olamazsın" diyebilir.
- **Dedicated vs. listen server:** Sunucu ayrı bir süreç mi, yoksa bir oyuncunun makinesinde mi?

Biz **authoritative client-server** seçtik çünkü:

- Çarpışma, patlama, ölüm gibi **kritik kararlar sunucuda** veriliyor → hile imkânsızlaşıyor.
- İstemciler sadece **görsel durumu** alıyor → tutarlılık garanti.
- İleride farklı cihazlara (mobil, web) kolayca istemci yazılabilir.

---

2. İki Protokolün Birleşimi: TCP + UDP

Bu projenin en öğretici kısmı, **aynı oyunda iki farklı taşıma katmanı** kullanmamızdır.

TCP (Transmission Control Protocol) — Port 5000

Ne için kullandık:

- Oyuncu katılımı (`join`)
- Oda oluşturma
- İsim/renk atama ve haritanın ilk gönderimi
- Oyundan ayrılma bildirimi (`leave`)

Neden TCP:

- TCP **güvenilir** (kaybolan paket yeniden gönderilir), **sıralı** (paketler geldiği sırada teslim edilir) ve **bağlantı yönelimli**dir (handshake ile).
- Katılım gibi işlemlerde **tek bir paketin kaybolması bile oyuncuyu oyundan koparır**. Örneğin "harita" paketi kaybolursa oyuncu boş ekran görür. TCP bunu garanti eder.
- Ayrıca TCP bağlantısı **açık kalır**; biz bunu "leave" mesajını dinlemek için kullandık. Bağlantı koptuğunda (`recv` boş döner) sunucu oyuncunun ayrıldığını anlar.

TCP'nin dezavantajı: Head-of-line blocking (bir paket kaybolursa sonrakiler bekler), yeniden gönderim gecikmesi. Bu yüzden **gerçek zamanlı konum güncellemeleri için uygun değil**.

UDP (User Datagram Protocol) — Port 5001

Ne için kullandık:

- Oyuncu hareketleri (30 Hz)
- Bomba bırakma, tekmeleme
- Patlama/alev durumu
- Power-up durumu
- Süre ve skor

Neden UDP:

- UDP **güvenilir değil, sırasız** — ama **hızlı** ve **düşük gecikmeli**.
- Bir konum paketi kaybolursa, **0.033 saniye sonra zaten yeni bir paket geliyor**. Eski paketi yeniden göndermenin anlamı yok — çünkü eski konum artık geçersiz.
- Bu, **"son durum kazanır" (last state wins)** prensibidir. Multiplayer FPS/RTS oyunlarının çoğu bu yüzden UDP kullanır.

Kritik kavram: Paket kaybı toleransı. UDP'de kayıp normaldir. Oyun tasarımı buna göre yapılmalı: her paket **tam durum (full state)** içermeli veya **idempotent** olmalı (iki kez uygulanması zarar vermemeli).

---

3. State (Durum) Yönetimi: Authoritative State

Ne yaptık: Sunucu `rooms` adlı global bir sözlükte tüm oyun durumunu tutuyor:

```
room = {
    "players": {...},    # oyuncular, konumlar, can, renk, efektler
    "map": [[...]],      # ızgara
    "bombs": [...],      # bombalar
    "flames": [...],     # aktif alevler
    "powerups": {...},   # yerdeki ödüller
    "state": "waiting",  # waiting / playing / finished
    "start_time": ...,
    "finished_at": ...,
    "result": {...},
}
```

Neden merkezi durum:

- Tüm istemciler **aynı gerçeği** görsün diye.
- İstemci kendi konumunu "tahmin" edebilir (client-side prediction), ama **sunucunun dediği son söz**.
- Hata ayıklama kolay: tek bir yerde tüm oyun var.

Kritik kavram: Authoritative vs. non-authoritative.

- Non-authoritative (istemci otoriteli): İstemci "ben buradayım" der, sunucu kabul eder → hile açık.

- Authoritative (sunucu otoriteli): İstemci "sola gitmek istiyorum" der, sunucu "tamam, sola gidebilirsin" der → hile kapalı.

Bizim tasarımımızda istemci **girdi (input)** gönderiyor, **konum** değil. Sunucu girdiyi işleyip konumu güncelliyor. Bu, doğru yaklaşımdır.

---

4. Serileştirme (Serialization): JSON

Ne yaptık: Tüm mesajları Python sözlüğü → `json.dumps()` → bayt dizisi → `sendto/sendall` şeklinde gönderdik. Karşı tarafta `json.loads()` ile geri çözdük.

Neden JSON:

- İnsan tarafından okunabilir (debug kolay).
- Dilden bağımsız (ileride JavaScript/C# istemci yazılabilir).
- Python'da standart kütüphanede.
- `pickle` **güvenlik riski** taşır: kötü niyetli bir paket rastgele kod çalıştırabilir.

JSON'un dezavantajı:

- **Büyük ve yavaş.** Her paket ~1-4 KB. 30 Hz × 4 oyuncu = saniyede ~120 KB. Gerçek oyunda bu **binary protokol** (protobuf, MessagePack, FlatBuffers) veya **elle paketlenmiş struct** ile yapılır. Örneğin bir oyuncu konumu 2 byte ile (x, y) gönderilebilir.
- **Sayısal verimlilik yok.** JSON'da `123` metin olarak gider (3 byte); binary'de `0x7B` (1 byte).

İleri seviye: `struct.pack('<BB', x, y)` ile 2 byte'lık konum paketi yazılabilir. Bu, bant genişliğini 10-50 kat azaltır. Projemizde öğretici olsun diye JSON kaldı; bir sonraki adımda binary'ye geçebiliriz.

---

5. Paket Tasarımı: Mesaj Türleri

Ne yaptık: Her mesajın bir `type` alanı var:

Tip	Protokol	Yön	Amaç
`join`	TCP	C→S	Odaya katıl, isim gönder
`joined`	TCP	S→C	Kabul onayı, ID/renk/harita
`leave`	TCP	C→S	Oyundan ayrıl
`input`	UDP	C→S	Hareket / bomba / tekme
`hello`	UDP	C→S	UDP adres eşleştirme
`state`	UDP	S→C	Tam oyun durumu

Kritik kavram: Protokol versiyonlama. Gerçek oyunda mesajların başına bir `version` alanı konur; eski istemci yeni sunucuyla konuşunca uyumsuzluk net anlaşılır.

Kritik kavram: Mesaj yönü ayrımı. Katılım gibi "bir kere olan" işlemler TCP; "sürekli tekrarlanan" işlemler UDP.

---

6. TCP'de "Bağlantıyı Açık Tut" Problemi ve Ayrı Thread

Ne yaptık (ve bir hatayı düzelttik): İlk denemede `handle_tcp_client` fonksiyonu `join` işlendikten sonra `while True: conn.recv()` döngüsünde bloke oluyordu. Bu, **ikinci istemcinin bağlanmasını engelliyordu** çünkü aynı thread içinde kalıyordu.

Çözüm: `join` işlendikten sonra bağlantıyı **ayrı bir thread'e** (`_listen_leave`) devrettik; ana `handle_tcp_client` thread'i hemen bitti. Böylece `tcp_sock.accept()` döngüsü hiçbir zaman bloke olmadı.

Kritik kavram: Thread-per-connection modeli. Her TCP bağlantısı kendi thread'inde. Bu, 10-100 oyuncu için uygundur. 1000+ için **async I/O** (`asyncio`, `select`, `epoll`) gerekir.

Kritik kavram: Deadlock'tan kaçınma. `room_lock` gibi kilitleri **kısa tut**; `sendall` gibi bloke edebilen çağrıları kilit dışında yap. Aksi halde bir istemci yavaşsa tüm sunucu kilitlenir.

---

7. UDP Adres Eşleştirme: Neden Ayrı Bir "hello" Paketi?

Ne yaptık: İstemci TCP ile katılıyor, ama UDP soketi farklı bir porttan (`bind(('0.0.0.0', 0))` → rastgele port) konuşuyor. Sunucu **hangi UDP adresinin hangi oyuncuya ait olduğunu** bilmiyor. Bu yüzden istemci `hello` paketi gönderiyor; sunucu `player_id` ile eşleştirip `udp_addr` alanını dolduruyor.

Neden gerekli: TCP bağlantısının kaynak adresi (IP:port) ile UDP soketinin kaynak adresi **farklıdır**. TCP'de 5000'e bağlanırsın, UDP'de rastgele port. Sunucu UDP paketinin hangi oyuncudan geldiğini anlamak için bu eşleştirmeye ihtiyaç duyar.

Kritik kavram: NAT (Network Address Translation). Gerçek internette evdeki router'lar IP'leri çevirir. UDP delik açma (UDP hole punching) bu yüzden karmaşıktır. Biz localhost'ta olduğumuz için sorun yok.

Kritik kavram: Paket kaybına dayanıklılık. `hello` paketini **3 kez** gönderdik çünkü UDP'de ilk paket kaybolabilir. Bu, "**retry with idempotency**" prensibidir.

---

8. Oyun Döngüsü: Tick Rate ve Broadcast

Ne yaptık: Sunucu `1/30 saniye` (30 Hz) aralıkla tüm istemcilere state yayınlıyor. İstemci de hareket girişini `0.12 saniye` aralıkla gönderiyor.

Neden tick rate önemli:

- **Çok düşük** (10 Hz) → hareket kesik kesik görünür.
- **Çok yüksek** (120 Hz) → bant genişliği ve CPU boşa gider.
- **30 Hz** çoğu 2D oyun için tatmin edici. FPS oyunları 60-128 Hz kullanır.

Kritik kavram: Fixed timestep vs. variable timestep.

- **Fixed:** Her tick'te `dt = 1/30`. Deterministik, tekrarlanabilir.
- **Variable:** Gerçek geçen süre. Gerçek zamana daha sadık ama deterministik değil.

Biz `dt = 1/30` sabit kullandık. Bomba süresi, alev süresi hep bu dt ile azaltılıyor.

Kritik kavram: Broadcast vs. unicast. Biz her oyuncuya **aynı state paketini** gönderiyoruz (broadcast). Daha verimli yol: **delta compression** (sadece değişen alanları gönder). Örneğin hareket etmeyen oyuncunun konumu tekrar gönderilmez. Bu, "**delta state**" veya "**snapshot + delta**" modelidir.

---

9. Gecikme ve Düzeltme Stratejileri

Bu projede **basit yaklaşım** kullandık: istemci sadece sunucunun gönderdiği konumu çizer. Ama gerçek oyunda gecikme (latency) sorun olur.

Bilinmesi gereken üç teknik:

a) Client-Side Prediction (İstemci Taraflı Tahmin)

İstemci tuşa bastığında **hemen** kendi karakterini hareket ettirir; sunucudan onay gelince düzeltir. Bu, algılanan gecikmeyi sıfıra indirir. Bizim projede yok; eklenebilir.

b) Server Reconciliation (Sunucu Uzlaştırması)

Sunucu "sen aslında burada değilsin, şuradasın" derse istemci karakteri oraya **yumuşakça** kaydırır. Ani zıplama (rubber-banding) olmaması için interpolasyon gerekir.

c) Entity Interpolation (Varlık İnterpolasyonu)

Diğer oyuncuların konumları 30 Hz gelir; istemci 60 FPS çizer. İki paket arasında **lineer interpolasyon** yaparak akıcı gösterir. Bizim projede bu yok; `self.players` doğrudan çiziliyor, bu yüzden 30 FPS gibi görünebilir.

Kritik kavram: Lag compensation. Yüksek pingli oyuncular için sunucu, "geçmişe dönük" çarpışma kontrolü yapar (ateş ettiğinde kurşunun geçmişteki konuma denk gelmesi). FPS oyunlarında standart; 2D Bomberman'da gerekmez.

---

10. Input Gönderimi: Neden Her Tuşta Değil, Basılı Tutunca?

Ne yaptık: Önce sadece `KEYDOWN` olayında hareket gönderiyorduk. Bu, tuşu basılı tutunca karakterin **tek kare** gitmesine yol açıyordu. Sonra `pygame.key.get_pressed()` ile **her 0.12 saniyede bir** hareket gönderdik.

Neden: Klavye otomatik tekrarı (key repeat) işletim sistemine bağlıdır, güvenilmez. Oyunlar genellikle **tuş durumunu (pressed/released)** takip edip kendi zamanlamasıyla hareket ettirir.

Kritik kavram: Input buffering. İstemci tuşa bastığında girişi **buffer'a** koyar; sunucu tick'inde buffer'daki girişleri işler. Bu, paket kaybına dayanıklılık sağlar. Bizim projede buffer yok; her paket anlık.

---

11. Yetkilendirme ve Doğrulama (Validation)

Ne yaptık: Sunucu her hareket isteğinde:

1. Hedef kare sınırlar içinde mi?

2. Haritada duvar var mı?
3. O karede bomba var mı?
4. Oyuncu yeterince bekledi mi (`MOVE_DELAY`)?
5. Oyuncu canlı mı?

Kontrollerinden geçerse konumu günceller.

Neden: İstemci "beni 100 kare ileri ışınla" dese bile sunucu reddeder. Bu, **server-side validation**'dır. Multiplayer'da **asla istemciye güvenme** prensibi.

Kritik kavram: Rate limiting. Sunucu, istemcinin saniyede kaç hareket gönderebileceğini sınırlar (`last_move` + `MOVE_DELAY`). Aksi halde bir hileci saniyede 1000 hareket gönderip ışınlanır.

---

12. Renk/İsim Atama: Sunucu Tarafı Tahsis

Ne yaptık: Renkleri sunucu `COLORS` listesinden sırayla atıyor. İsim çakışmasını `base_name + suffix` ile çözüyor.

Neden sunucu tarafı:

- İstemci kendi rengini seçseydi, iki oyuncu aynı rengi seçebilirdi.
- İsim çakışması da benzer şekilde sunucuda çözülmeli.
- **Deterministik tahsis**: aynı sırayla katılan oyuncular hep aynı rengi alır.

---

13. Bağlantı Kopması ve "Zombie Oyuncu" Problemi

Ne yaptık: Önceki durumda pencereyi kapatan oyuncu sunucuda "hâlâ var" görünüyordu. Çözüm:

- İstemci pencere kapanırken/sağ üstteki **ÇIKIŞ** butonunda `leave` mesajı gönderiyor.
- Sunucu `_listen_leave` thread'inde `recv` boş dönerse (TCP koptu) **otomatik** oyuncuyu siliyor.

Neden: TCP bağlantısı koptuğunda sunucu bunu **hemen** anlar (`recv` 0 döner). Ama UDP'de bu sinyal yok — bu yüzden **TCP'yi "canlılık" (liveness) için kullanmak** yaygın bir tekniktir.

Kritik kavram: Timeout / heartbeat. UDP-only bir sistemde her istemci belirli aralıkla `ping` gönderir; sunucu 5 saniye paket almazsa oyuncuyu atar. Bizim projede TCP bunu doğal olarak sağlıyor.

---

14. Oyun Durumu Geçişleri (State Machine)

Ne yaptık: Oda üç durum arasında geçiyor:

```
waiting ──(2+ oyuncu)──> playing ──(süre bitti / tek canlı)──> finished
   ↑                                                              │
   └──────────────────(10s sonra, 2+ oyuncu varsa)────────────────┘
   └──────────────────(oyuncu ayrıldıysa, <2 oyuncu)──────────────┘
```

Neden state machine:

- Her durumda farklı kurallar geçerli (waiting'de hareket yok, playing'de var, finished'de sadece gösterim).
- Geçişler **açık ve test edilebilir**.
- Bug'ları önler: "finished" durumunda bomba patlaması gibi tutarsızlıkları engeller.

Kritik kavram: Reset (sıfırlama). Oyun yeniden başlarken `reset_room` **tüm geçici durumu** sıfırlar: bomba listesi, alev listesi, power-up listesi, harita, oyuncu efektleri, süre. **Kalıcı olması gereken** ise (oyuncu ismi, rengi) korunur.

---

15. Power-up Sistemi: Etki ve Süre Yönetimi

Ne yaptık:

- Power-up'lar **kırılan blok altından** çıkar (rastgele).
- Oyuncu üzerine basınca aktif olur.
- **Geçici etkiler** `effects` sözlüğünde bitiş zamanı ile tutulur: `{'speed': now + 8.0}`.
- **Kalıcı etkiler** (extra_bomb, range) doğrudan oyuncu alanına yazılır.
- Sunucu her tick'te süresi dolanları temizler.

Neden bu tasarım:

- **Merkezi süre**: tüm etkiler sunucuda; istemci sadece kalan süreyi gösterir.
- **Deterministik**: aynı tick'te süre dolarsa aynı anda biter.
- **Birleşebilirlik**: aynı anda birden çok etki olabilir (hız + menzil).

Kritik kavram: Buff/debuff stacking. Gerçek oyunda "hız" iki kez alınırsa ne olur? Bizde üst üste yazılır (son süre kazanır). Daha karmaşık sistemlerde **stack limit** ve **süre uzatma** kuralları vardır.

---

16. Güvenlik Notları

Projemiz localhost'ta eğitim amaçlı olduğu için bazı açıklar var. Gerçek oyunda:

Açık	Çözüm
UDP paketleri şifrelenmemiş	DTLS / QUIC
Kimlik doğrulama yok (`player_id` tahmin edilebilir)	Token / session ID
`player_id` istemci tarafından gönderiliyor, sunucu güveniyor	Sunucu eşleştirmeyi `udp_addr` ile yapmalı
JSON deserialization'da boyut sınırı yok	`recvfrom(4096)` sınırı var ama daha katı olmalı
Rate limiting sadece hareket için	Tüm mesaj türleri için

Kritik kavram: Never trust the client. İstemcinin gönderdiği **hiçbir veriye** doğrudan güvenme. `player_id`'yi bile sunucu **UDP adres eşleşmesi** ile doğrulamalı (bizde kısmen öyle).

---

17. Ölçeklenebilirlik: Şu An Neredeyiz?

Şu anki tasarım **küçük ölçek** için uygun:

- 8 oyuncuya kadar tek oda.
- Tüm istemcilere broadcast.
- JSON serileştirme.

Büyük ölçek için gerekenler:

- **Room-based sharding**: her oda ayrı süreç/thread.
- **Binary protocol** (protobuf/MessagePack).
- **Delta compression**: sadece değişen alanlar.
- **Interest management**: her istemci sadece **görebildiği** varlıkları alır (görüş alanı dışındakiler gönderilmez).
- **Load balancer / matchmaker**: oyuncuları uygun sunucuya yönlendirme.
- **Async I/O** (asyncio) — thread-per-connection 10k+ için uygun değil.

---

18. Öğrendiğimiz Kavramların Özeti

Kavram	Projedeki Karşılığı
Client-server	Sunucu otorite, istemciler görüntüleyici
TCP	Katılım, ayrılma, harita (güvenilir, sıralı)
UDP	Konum, bomba, alev (hızlı, kayıp toleranslı)
Serialization	JSON (`dumps`/`loads`)
State management	Merkezi `rooms` sözlüğü
State machine	`waiting → playing → finished`
Tick rate	30 Hz broadcast
Broadcast	Her istemciye tam state
Validation	Sunucu tarafı çarpışma/rate limit
Rate limiting	`last_move` + `MOVE_DELAY`
Thread-per-connection	Her TCP istemcisi ayrı thread
Retry	`hello` 3 kez
UDP eşleştirme	`hello` + `player_id`
Timeout / liveness	TCP kopunca oyuncu silinir
Reset	`reset_room` tüm geçici durumu sıfırlar
Buff/debuff	`effects` sözlüğü + bitiş zamanı
Authoritative simulation	Sunucu tüm oyun mantığını çalıştırır

Kavram	Projedeki Karşılığı
Server-side validation	İstemciye güvenme prensibi

---

19. Bir Sonraki Adımda Neler Eklenebilir?

1. **Client-side prediction** — hareket gecikmesini sıfıra indirmek.
2. **Entity interpolation** — diğer oyuncuları akıcı göstermek.
3. **Binary protocol** — `struct.pack` ile bant genişliğini 10× azaltmak.
4. **Heartbeat / ping** — UDP-only liveness.
5. **Reconnect** — kopan oyuncunun geri dönmesi.
6. **Spectator mode** — ölen oyuncunun izlemesi.
7. **Multiple rooms** — matchmaking.
8. **Sunucu tarafı AI** — bot oyuncular.
9. **Chat sistemi** — TCP üzerinden.
10. **Kayıt/replay** — state geçmişini saklayıp geri oynatma.

Oyun kodları:

```
//server.py

# server.py
import socket
import threading
import json
import time
import random

TCP_PORT = 5000
UDP_PORT = 5001
HOST = '127.0.0.1'
GRID_SIZE = 15
GAME_DURATION = 60.0
RESTART_DELAY = 10.0
BOMB_FUSE = 3.0

MOVE_DELAY_BASE = 0.14
MOVE_DELAY_FAST = 0.07
MOVE_DELAY_SLOW = 0.22

POWERUP_TYPES = ['speed', 'extra_bomb', 'slow', 'range', 'kick', 'reverse']
POWERUP_DURATION = {'speed': 8.0, 'slow': 6.0, 'kick': 10.0, 'reverse':
5.0}

COLORS = [
    (0, 200, 0), (200, 0, 0), (0, 100, 255), (255, 200, 0),
    (255, 0, 255), (0, 255, 255), (255, 140, 0), (150, 0, 255),
]

rooms = {}
```

```
room_lock = threading.Lock()

def log(*args):
    print("[SERVER]", *args, flush=True)

def generate_map():
    grid = [[0] * GRID_SIZE for _ in range(GRID_SIZE)]
    for i in range(GRID_SIZE):
        for j in range(GRID_SIZE):
            if i == 0 or i == GRID_SIZE - 1 or j == 0 or j == GRID_SIZE -
1:
                grid[i][j] = 1
            elif i % 2 == 0 and j % 2 == 0:
                grid[i][j] = 1
            elif random.random() < 0.35:
                grid[i][j] = 2
    for (sx, sy) in [(1,1),(1,13),(13,1),(13,13),(7,7),(1,7),(13,7),(7,1)]:
        grid[sy][sx] = 0
    return grid

def pick_spawn(room):
    candidates = [(1,1),(1,13),(13,1),(13,13),(7,7),(1,7),(13,7),(7,1)]
    used = {(p['x'], p['y']) for p in room['players'].values() if
p['alive']}
    for (x, y) in candidates:
        if (x, y) not in used:
            return x, y
    return random.randint(1, GRID_SIZE-2), random.randint(1, GRID_SIZE-2)

def reset_room(room):
    room['map'] = generate_map()
    room['bombs'] = []
    room['flames'] = []
    room['powerups'] = {}
    room['result'] = None
    room['start_time'] = None
    room['finished_at'] = None

    used_positions = set()
    for pid, p in room['players'].items():
        candidates = [(1,1),(1,13),(13,1),(13,13),(7,7),(1,7),(13,7),(7,1)]
        pos = None
        for (x, y) in candidates:
            if (x, y) not in used_positions:
                pos = (x, y)
                break
        if pos is None:
            pos = (random.randint(1, GRID_SIZE-2), random.randint(1,
GRID_SIZE-2))
        used_positions.add(pos)
        p['x'], p['y'] = pos
        p['alive'] = True
        p['bomb_count'] = 1
        p['flame_range'] = 1
```

```
            p['active_bombs'] = 0
            p['effects'] = {}
            p['last_move'] = 0.0

        if len(room['players']) >= 2:
            room['state'] = 'playing'
            room['start_time'] = time.time()
            log(f"Oyun başladı. Oyuncular: {[p['name'] for p in
room['players'].values()]}")
        else:
            room['state'] = 'waiting'
            log(f"Waiting. Oyuncu sayısı: {len(room['players'])}")

def _listen_leave(conn, addr, pid):
    """Ayrı thread: TCP bağlantısını dinle, leave gelince oyuncuyu sil."""
    try:
        conn.settimeout(1.0)
        while True:
            try:
                data = conn.recv(4096)
            except socket.timeout:
                continue
            except:
                break
            if not data:
                break
            try:
                msg = json.loads(data.decode('utf-8'))
            except:
                continue
            if msg.get('type') == 'leave':
                log(f"Leave mesajı alındı: {pid}")
                break
    finally:
        with room_lock:
            for rid, room in rooms.items():
                if pid in room['players']:
                    name = room['players'][pid]['name']
                    del room['players'][pid]
                    log(f"{name} ({pid}) ayrıldı. Kalan:
{len(room['players'])}")
                    # Oyun sırasında ayrıldıysa durum güncelle
                    if room['state'] == 'playing':
                        alive = [p for p in room['players'].values() if
p['alive']]
                        if len(room['players']) < 2:
                            if len(alive) == 1:
                                room['result'] = {"winner": alive[0]['id'],
"reason": "opponent_left"}
                            else:
                                room['result'] = {"winner": None, "reason":
"draw"}
                            room['state'] = 'finished'
                            room['finished_at'] = time.time()
```

```
                    # 2'den az oyuncu → waiting
                    if len(room['players']) < 2:
                        room['state'] = 'waiting'
                        room['start_time'] = None
                        room['bombs'] = []
                        room['flames'] = []
                        room['powerups'] = {}
                        room['map'] = generate_map()
                        room['result'] = None
                        room['finished_at'] = None
                    break
        try:
            conn.close()
        except:
            pass

def handle_tcp_client(conn, addr):
    pid = f"{addr[0]}:{addr[1]}"
    try:
        conn.settimeout(5.0)
        data = conn.recv(4096).decode('utf-8')
        if not data:
            log(f"Boş veri: {addr}")
            conn.close()
            return
        msg = json.loads(data)
        log(f"TCP mesaj: {msg.get('type')} from {addr}")
        if msg['type'] != 'join':
            conn.close()
            return

        room_id = msg.get('room_id', 'default')
        name = msg.get('name', 'Oyuncu')[:16]

        with room_lock:
            if room_id not in rooms:
                rooms[room_id] = {
                    "players": {}, "map": generate_map(), "bombs": [],
"flames": [],
                    "powerups": {}, "state": "waiting", "start_time": None,
                    "finished_at": None, "result": None,
                }
            room = rooms[room_id]

            existing_names = {p['name'] for p in room['players'].values()}
            base_name = name
            suffix = 1
            while name in existing_names:
                name = f"{base_name}{suffix}"
                suffix += 1

            used_colors = {tuple(p['color']) for p in
room['players'].values()}
            color = None
```

```
            for c in COLORS:
                if c not in used_colors:
                    color = c
                    break
            if color is None:
                color = COLORS[len(room['players']) % len(COLORS)]

            spawn_x, spawn_y = pick_spawn(room)

            room['players'][pid] = {
                "id": pid, "name": name, "x": spawn_x, "y": spawn_y,
                "alive": True, "color": list(color),
                "bomb_count": 1, "flame_range": 1, "active_bombs": 0,
                "udp_addr": None, "effects": {}, "last_move": 0.0,
            }

            response = {
                "type": "joined", "room_id": room_id, "player_id": pid,
                "name": name, "color": list(color), "map": room['map'],
            }
            conn.sendall(json.dumps(response).encode('utf-8'))
            log(f"{name} katıldı. Odadaki oyuncu sayısı:
{len(room['players'])}")

            # 2+ oyuncu varsa oyunu başlat
            if len(room['players']) >= 2 and room['state'] == 'waiting':
                reset_room(room)

        # Bağlantıyı ayrı thread'de dinle (leave için)
        threading.Thread(target=_listen_leave, args=(conn, addr, pid),
daemon=True).start()

    except Exception as e:
        log(f"TCP Hata ({addr}): {e}")
        try:
            conn.close()
        except:
            pass

def udp_game_loop():
    udp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    udp_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    udp_sock.bind((HOST, UDP_PORT))
    log(f"UDP Dinleniyor: {HOST}:{UDP_PORT}")

    last_broadcast = 0.0
    BROADCAST_INTERVAL = 1.0 / 30.0

    while True:
        udp_sock.settimeout(0.005)
        try:
            while True:
                data, addr = udp_sock.recvfrom(4096)
                try:
```

```
                    msg = json.loads(data.decode('utf-8'))
                except:
                    continue

                with room_lock:
                    target_room = None
                    target_pid = None
                    for rid, room in rooms.items():
                        for pid, p in room['players'].items():
                            if p['udp_addr'] == addr:
                                target_room = room
                                target_pid = pid
                                break
                        if target_room:
                            break

                    if target_room is None and 'player_id' in msg:
                        pid_hint = msg['player_id']
                        for rid, room in rooms.items():
                            if pid_hint in room['players']:
                                if room['players'][pid_hint]['udp_addr'] is
None:
                                    room['players'][pid_hint]['udp_addr'] =
addr
                                    log(f"UDP addr eşleşti: {pid_hint} ->
{addr}")
                                target_room = room
                                target_pid = pid_hint
                                break

                    if not (target_room and target_pid):
                        continue

                    room = target_room
                    player = room['players'][target_pid]
                    if room['state'] != 'playing' or not player['alive']:
                        continue

                    now = time.time()
                    action = msg.get('action')

                    if action == 'move':
                        delay = MOVE_DELAY_BASE
                        if 'speed' in player['effects'] and
player['effects']['speed'] > now:
                            delay = MOVE_DELAY_FAST
                        if 'slow' in player['effects'] and
player['effects']['slow'] > now:
                            delay = MOVE_DELAY_SLOW
                        if now - player['last_move'] < delay:
                            continue
                        player['last_move'] = now

                        dx, dy = msg.get('dx', 0), msg.get('dy', 0)
```

```
                        if 'reverse' in player['effects'] and
player['effects']['reverse'] > now:
                            dx, dy = -dx, -dy
                        if dx == 0 and dy == 0:
                            continue
                        nx, ny = player['x'] + dx, player['y'] + dy
                        blocked = False
                        if not (0 <= nx < GRID_SIZE and 0 <= ny <
GRID_SIZE):
                            blocked = True
                        elif room['map'][ny][nx] != 0:
                            blocked = True
                        elif any(b['x'] == nx and b['y'] == ny for b in
room['bombs']):
                            blocked = True
                        if not blocked:
                            player['x'], player['y'] = nx, ny
                            if (nx, ny) in room['powerups']:
                                ptype = room['powerups'].pop((nx, ny))
                                apply_powerup(player, ptype, now)

                    elif action == 'bomb':
                        if player['active_bombs'] < player['bomb_count']:
                            if not any(b['x'] == player['x'] and b['y'] ==
player['y'] for b in room['bombs']):
                                room['bombs'].append({
                                    "x": player['x'], "y": player['y'],
                                    "range": player['flame_range'],
                                    "owner": target_pid,
                                    "timer": BOMB_FUSE, "max_timer":
BOMB_FUSE,
                                })
                                player['active_bombs'] += 1

                    elif action == 'kick':
                        if 'kick' in player['effects'] and
player['effects']['kick'] > now:
                            dx, dy = msg.get('dx', 0), msg.get('dy', 0)
                            if 'reverse' in player['effects'] and
player['effects']['reverse'] > now:
                                dx, dy = -dx, -dy
                            tx, ty = player['x'] + dx, player['y'] + dy
                            for b in room['bombs']:
                                if b['x'] == tx and b['y'] == ty:
                                    nx2, ny2 = tx + dx, ty + dy
                                    while (0 <= nx2 < GRID_SIZE and 0 <=
ny2 < GRID_SIZE
                                           and room['map'][ny2][nx2] == 0
                                           and not any(bb['x'] == nx2 and
bb['y'] == ny2 for bb in room['bombs'])):
                                        b['x'], b['y'] = nx2, ny2
                                        nx2, ny2 = nx2 + dx, ny2 + dy
                                    break
        except socket.timeout:
```

```
            pass

        with room_lock:
            now = time.time()
            for rid, room in list(rooms.items()):
                if room['state'] == 'finished':
                    if room['finished_at'] and now - room['finished_at'] >=
RESTART_DELAY:
                        if len(room['players']) >= 2:
                            log(f"Oda {rid} yeniden başlıyor.")
                            reset_room(room)
                        else:
                            room['state'] = 'waiting'
                            room['result'] = None
                            room['finished_at'] = None
                            room['map'] = generate_map()
                            room['bombs'] = []
                            room['flames'] = []
                            room['powerups'] = {}
                    continue

                if room['state'] != 'playing':
                    continue

                dt = 1.0 / 30.0
                elapsed = now - room['start_time']
                if elapsed >= GAME_DURATION:
                    alive_players = [p for p in room['players'].values() if
p['alive']]
                    if len(alive_players) == 1:
                        room['result'] = {"winner": alive_players[0]['id'],
"reason": "time"}
                    else:
                        room['result'] = {"winner": None, "reason": "draw"}
                    room['state'] = 'finished'
                    room['finished_at'] = now
                    continue

                for bomb in room['bombs'][:]:
                    bomb['timer'] -= dt
                    if bomb['timer'] <= 0:
                        bx, by, r = bomb['x'], bomb['y'], bomb['range']
                        flames = [(bx, by)]
                        for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
                            for i in range(1, r+1):
                                nx, ny = bx + dx*i, by + dy*i
                                if not (0 <= nx < GRID_SIZE and 0 <= ny <
GRID_SIZE):
                                    break
                                if room['map'][ny][nx] == 1:
                                    break
                                flames.append((nx, ny))
                                if room['map'][ny][nx] == 2:
                                    room['map'][ny][nx] = 0
```

```
                                    if random.random() < 0.5:
                                        room['powerups'][(nx, ny)] =
random.choice(POWERUP_TYPES)
                                    break
                        room['flames'].append({"cells": flames, "timer":
0.5})

                        for pid, p in room['players'].items():
                            if p['alive'] and (p['x'], p['y']) in flames:
                                p['alive'] = False
                                p['effects'] = {}
                                log(f"{p['name']} patladı.")

                        owner = bomb.get('owner')
                        if owner and owner in room['players']:
                            room['players'][owner]['active_bombs'] = max(
                                0, room['players'][owner]['active_bombs'] -
1)

                        room['bombs'].remove(bomb)

                for flame in room['flames'][:]:
                    flame['timer'] -= dt
                    if flame['timer'] <= 0:
                        room['flames'].remove(flame)

                alive_players = [p for p in room['players'].values() if
p['alive']]
                if len(room['players']) >= 2 and len(alive_players) <= 1:
                    if len(alive_players) == 1:
                        room['result'] = {"winner": alive_players[0]['id'],
"reason": "elimination"}
                    else:
                        room['result'] = {"winner": None, "reason": "draw"}
                    room['state'] = 'finished'
                    room['finished_at'] = now

        if now - last_broadcast >= BROADCAST_INTERVAL:
            with room_lock:
                # Toplanacak adresler: her odadaki oyuncuların
udp_addr'leri
                all_addrs = []
                for rid, room in rooms.items():
                    for pid, p in room['players'].items():
                        if p['udp_addr'] is not None:
                            all_addrs.append(p['udp_addr'])

                for rid, room in rooms.items():
                    remaining = 0
                    if room['start_time'] is not None and room['state'] ==
'playing':
                        remaining = max(0, GAME_DURATION - (time.time() -
room['start_time']))
                    restart_in = 0
```

```
                    if room['state'] == 'finished' and room['finished_at']:
                        restart_in = max(0, RESTART_DELAY - (time.time() -
room['finished_at']))

                    powerups_list = [{"x": x, "y": y, "type": t} for (x,
y), t in room['powerups'].items()]
                    players_out = {}
                    for pid, p in room['players'].items():
                        active_fx = {k: max(0, v - now) for k, v in
p['effects'].items() if v > now}
                        players_out[pid] = {
                            "name": p['name'], "x": p['x'], "y": p['y'],
                            "alive": p['alive'], "color": p['color'],
                            "bomb_count": p['bomb_count'], "flame_range":
p['flame_range'],
                            "active_bombs": p['active_bombs'], "effects":
active_fx,
                        }

                    state_msg = {
                        "type": "state", "state": room['state'],
                        "remaining": remaining, "restart_in": restart_in,
                        "players": players_out,
                        "bombs": room['bombs'],
                        "flames": [f['cells'] for f in room['flames']],
                        "powerups": powerups_list,
                        "map": room['map'],
                        "result": room['result'],
                    }
                    data = json.dumps(state_msg).encode('utf-8')
                    for ca in all_addrs:
                        try:
                            udp_sock.sendto(data, ca)
                        except:
                            pass
            last_broadcast = now

def apply_powerup(player, ptype, now):
    if ptype == 'speed':
        player['effects']['speed'] = now + POWERUP_DURATION['speed']
    elif ptype == 'slow':
        player['effects']['slow'] = now + POWERUP_DURATION['slow']
    elif ptype == 'kick':
        player['effects']['kick'] = now + POWERUP_DURATION['kick']
    elif ptype == 'reverse':
        player['effects']['reverse'] = now + POWERUP_DURATION['reverse']
    elif ptype == 'extra_bomb':
        player['bomb_count'] += 1
    elif ptype == 'range':
        player['flame_range'] += 1
    log(f"{player['name']} -> {ptype}")

if __name__ == '__main__':
    tcp_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
    tcp_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    tcp_sock.bind((HOST, TCP_PORT))
    tcp_sock.listen(10)
    log(f"TCP Dinleniyor: {HOST}:{TCP_PORT}")
    threading.Thread(target=udp_game_loop, daemon=True).start()
    while True:
        conn, addr = tcp_sock.accept()
        log(f"TCP bağlantı: {addr}")
        threading.Thread(target=handle_tcp_client, args=(conn, addr),
daemon=True).start()
```

```
//client.py
//aynı client kodu ile birden fazla client oluşturulmuştur.

# client.py
import socket
import threading
import json
import time
import pygame

SERVER_HOST = '127.0.0.1'
TCP_PORT = 5000
UDP_PORT = 5001
GRID_SIZE = 15
CELL_SIZE = 40
INFO_BAR_HEIGHT = 60
WIDTH = GRID_SIZE * CELL_SIZE
HEIGHT = GRID_SIZE * CELL_SIZE + INFO_BAR_HEIGHT

BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
GRAY = (100, 100, 100)
BROWN = (139, 69, 19)
YELLOW = (255, 255, 0)
ORANGE = (255, 140, 0)
RED = (220, 30, 30)
DARK_GRAY = (40, 40, 40)
BUTTON_COLOR = (180, 40, 40)
BUTTON_HOVER = (220, 60, 60)

POWERUP_COLORS = {
    'speed': (255, 255, 0), 'extra_bomb': (0, 200, 255),
    'slow': (150, 150, 150), 'range': (255, 100, 0),
    'kick': (200, 100, 255), 'reverse': (255, 0, 100),
}
POWERUP_LABELS =
{'speed':'H','extra_bomb':'B','slow':'Y','range':'M','kick':'T','reverse':'
R'}
```

```
class Client:
    def __init__(self, name):
        self.player_id = None
        self.player_name = name
        self.map_data = None
        self.players = {}
        self.bombs = []
        self.flames = []
        self.powerups = {}
        self.state = "connecting"
        self.remaining = 0.0
        self.restart_in = 0.0
        self.result = None
        self.running = True
        self.last_move_time = 0.0
        self.last_dir = (0, 1)

        self.tcp_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.tcp_sock.connect((SERVER_HOST, TCP_PORT))
        join_msg = json.dumps({"type": "join", "room_id": "default",
"name": name}).encode('utf-8')
        self.tcp_sock.sendall(join_msg)

        response = self.tcp_sock.recv(8192).decode('utf-8')
        data = json.loads(response)
        self.player_id = data['player_id']
        self.player_name = data['name']
        self.map_data = data['map']
        print(f"Bağlandı. ID: {self.player_id} | İsim: {self.player_name}",
flush=True)

        self.udp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.udp_sock.bind(('0.0.0.0', 0))
        self.server_udp_addr = (SERVER_HOST, UDP_PORT)

        # UDP dinleme
        threading.Thread(target=self.udp_receive_loop, daemon=True).start()

        # hello paketini birkaç kez gönder (kaybolma ihtimaline karşı)
        for _ in range(3):
            self.send_input('hello')
            time.sleep(0.05)

        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption(f"Bomberman - {self.player_name}")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("arial", 16)
        self.small_font = pygame.font.SysFont("arial", 13)
        self.mid_font = pygame.font.SysFont("arial", 28, bold=True)
        self.big_font = pygame.font.SysFont("arial", 44, bold=True)

    def udp_receive_loop(self):
        while self.running:
```

```
            try:
                self.udp_sock.settimeout(0.1)
                data, _ = self.udp_sock.recvfrom(16384)
                msg = json.loads(data.decode('utf-8'))
                if msg['type'] == 'state':
                    self.state = msg.get('state', 'playing')
                    self.players = msg.get('players', {})
                    self.bombs = msg.get('bombs', [])
                    self.flames = msg.get('flames', [])
                    self.remaining = msg.get('remaining', 0)
                    self.restart_in = msg.get('restart_in', 0)
                    self.result = msg.get('result')
                    m = msg.get('map')
                    if m:
                        self.map_data = m
                    self.powerups = {}
                    for pu in msg.get('powerups', []):
                        self.powerups[(pu['x'], pu['y'])] = pu['type']
            except socket.timeout:
                pass
            except Exception as e:
                if self.running:
                    print(f"UDP hata: {e}", flush=True)

    def send_input(self, action, **kwargs):
        msg = {"type": "input", "action": action, "player_id":
self.player_id}
        msg.update(kwargs)
        try:
            self.udp_sock.sendto(json.dumps(msg).encode('utf-8'),
self.server_udp_addr)
        except:
            pass

    def leave(self):
        try:
            self.tcp_sock.sendall(json.dumps({"type":
"leave"}).encode('utf-8'))
        except:
            pass
        self.running = False

    def run(self):
        while self.running:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    self.leave()
                elif event.type == pygame.MOUSEBUTTONDOWN:
                    if event.button == 1 and
self.exit_button_rect().collidepoint(event.pos):
                        self.leave()
                elif event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_ESCAPE:
                        self.leave()
```

```
                    elif self.state == 'playing':
                        if event.key in (pygame.K_UP, pygame.K_w):
                            self.send_input('move', dx=0, dy=-1);
self.last_dir=(0,-1)
                        elif event.key in (pygame.K_DOWN, pygame.K_s):
                            self.send_input('move', dx=0, dy=1);
self.last_dir=(0,1)
                        elif event.key in (pygame.K_LEFT, pygame.K_a):
                            self.send_input('move', dx=-1, dy=0);
self.last_dir=(-1,0)
                        elif event.key in (pygame.K_RIGHT, pygame.K_d):
                            self.send_input('move', dx=1, dy=0);
self.last_dir=(1,0)
                        elif event.key == pygame.K_SPACE:
                            self.send_input('bomb')
                        elif event.key == pygame.K_x:
                            d = self.last_dir
                            self.send_input('kick', dx=d[0], dy=d[1])

            if self.state == 'playing':
                keys = pygame.key.get_pressed()
                now = time.time()
                if now - self.last_move_time > 0.12:
                    dx = dy = 0
                    if keys[pygame.K_UP] or keys[pygame.K_w]: dy = -1
                    elif keys[pygame.K_DOWN] or keys[pygame.K_s]: dy = 1
                    elif keys[pygame.K_LEFT] or keys[pygame.K_a]: dx = -1
                    elif keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx = 1
                    if dx or dy:
                        self.last_dir = (dx, dy)
                        self.send_input('move', dx=dx, dy=dy)
                        self.last_move_time = now

            self.draw()
            self.clock.tick(60)

        pygame.quit()
        try: self.tcp_sock.close()
        except: pass
        self.udp_sock.close()

    def exit_button_rect(self):
        return pygame.Rect(WIDTH - 90, 8, 80, 32)

    def draw(self):
        self.screen.fill(BLACK)
        pygame.draw.rect(self.screen, DARK_GRAY, (0, 0, WIDTH,
INFO_BAR_HEIGHT))
        self.draw_info_bar()
        self.draw_exit_button()

        offset_y = INFO_BAR_HEIGHT
        if self.map_data:
            for y in range(GRID_SIZE):
```

```
                for x in range(GRID_SIZE):
                    cell = self.map_data[y][x]
                    rect = pygame.Rect(x * CELL_SIZE, offset_y + y *
CELL_SIZE, CELL_SIZE, CELL_SIZE)
                    if cell == 1: pygame.draw.rect(self.screen, GRAY, rect)
                    elif cell == 2: pygame.draw.rect(self.screen, BROWN,
rect)
                    else: pygame.draw.rect(self.screen, BLACK, rect)
                    pygame.draw.rect(self.screen, (30,30,30), rect, 1)

        for (px, py), ptype in self.powerups.items():
            rect = pygame.Rect(px*CELL_SIZE+4, offset_y+py*CELL_SIZE+4,
CELL_SIZE-8, CELL_SIZE-8)
            color = POWERUP_COLORS.get(ptype, WHITE)
            pygame.draw.rect(self.screen, color, rect)
            pygame.draw.rect(self.screen, WHITE, rect, 2)
            label = self.small_font.render(POWERUP_LABELS.get(ptype, '?'),
True, BLACK)
            lr = label.get_rect(center=rect.center)
            self.screen.blit(label, lr)

        for flame_cells in self.flames:
            for (fx, fy) in flame_cells:
                rect = pygame.Rect(fx*CELL_SIZE, offset_y+fy*CELL_SIZE,
CELL_SIZE, CELL_SIZE)
                pygame.draw.rect(self.screen, ORANGE, rect)
                pygame.draw.rect(self.screen, (255,220,100), rect, 2)

        for bomb in self.bombs:
            bx = bomb['x']*CELL_SIZE; by = offset_y+bomb['y']*CELL_SIZE
            cx, cy = bx+CELL_SIZE//2, by+CELL_SIZE//2
            timer = bomb.get('timer', 0); max_timer = bomb.get('max_timer',
3.0)
            blink_speed = 6.0 if timer > max_timer*0.5 else 14.0
            visible = int(timer*blink_speed) % 2 == 0
            if visible:
                pygame.draw.circle(self.screen, (30,30,30), (cx,cy),
CELL_SIZE//2-6)
                pygame.draw.circle(self.screen, (60,60,60), (cx-4,cy-4), 5)
                pygame.draw.circle(self.screen, RED, (cx,cy), CELL_SIZE//2-
6, 2)
            pygame.draw.line(self.screen, (200,200,200), (cx+8,cy-10),
(cx+14,cy-18), 2)
            pygame.draw.circle(self.screen, YELLOW, (cx+14,cy-18), 3)

        for pid, pdata in self.players.items():
            if not pdata['alive']: continue
            color = tuple(pdata['color'])
            px = pdata['x']*CELL_SIZE+4; py =
offset_y+pdata['y']*CELL_SIZE+4
            rect = pygame.Rect(px, py, CELL_SIZE-8, CELL_SIZE-8)
            pygame.draw.rect(self.screen, color, rect)
            pygame.draw.rect(self.screen, WHITE, rect, 2)
            effects = pdata.get('effects', {})
```

```
            if effects:
                ex = px+2
                for fx_name in effects:
                    fc = POWERUP_COLORS.get(fx_name, WHITE)
                    pygame.draw.circle(self.screen, fc, (ex+3, py-4), 3)
                    ex += 8
            name_surf = self.small_font.render(pdata['name'], True, WHITE)
            self.screen.blit(name_surf, (px+2, py-16))

        if self.state == 'waiting':
            self.draw_overlay_text(["Diğer oyuncular bekleniyor...", "Oyun
2 oyuncu ile başlar"], WHITE)
        elif self.state == 'finished' and self.result:
            winner = self.result.get('winner')
            if winner is None: msg, color = "BERABERE", YELLOW
            elif winner == self.player_id: msg, color = "KAZANDIN!",
(0,255,0)
            else:
                winner_name = self.players.get(winner, {}).get('name', '?')
                msg, color = f"KAYBETTİN - Kazanan: {winner_name}", RED
            lines = [msg]
            if self.restart_in > 0:
                lines.append(f"Yeniden başlıyor:
{int(self.restart_in)+1}s")
            self.draw_overlay_text(lines, color)

        pygame.display.flip()

    def draw_info_bar(self):
        time_surf = self.font.render(f"Süre: {int(self.remaining)}s", True,
WHITE)
        self.screen.blit(time_surf, (10, 6))
        me = self.players.get(self.player_id)
        if me:
            fx = me.get('effects', {})
            info = f"B:{me['bomb_count']} M:{me['flame_range']}"
            if fx: info += " | " + " ".join(f"{k}:{int(v)}s" for k,v in
fx.items())
            surf = self.small_font.render(info, True, tuple(me['color']))
            self.screen.blit(surf, (10, 30))
        x = 140; y = 6
        for pid, pdata in self.players.items():
            color = tuple(pdata['color']) if pdata['alive'] else (80,80,80)
            pygame.draw.rect(self.screen, color, (x, y, 14, 14))
            pygame.draw.rect(self.screen, WHITE, (x, y, 14, 14), 1)
            label = pdata['name'] if pdata['alive'] else f"{pdata['name']}
(öldü)"
            surf = self.small_font.render(label, True, color)
            self.screen.blit(surf, (x+20, y))
            x += 20 + surf.get_width() + 20
            if x > WIDTH - 120: x = 140; y += 20

    def draw_exit_button(self):
        rect = self.exit_button_rect()
```

```
            mouse = pygame.mouse.get_pos()
            color = BUTTON_HOVER if rect.collidepoint(mouse) else BUTTON_COLOR
            pygame.draw.rect(self.screen, color, rect)
            pygame.draw.rect(self.screen, WHITE, rect, 2)
            label = self.font.render("ÇIKIŞ", True, WHITE)
            lr = label.get_rect(center=rect.center)
            self.screen.blit(label, lr)

        def draw_overlay_text(self, lines, color):
            overlay = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
            overlay.fill((0,0,0,160))
            self.screen.blit(overlay, (0,0))
            font = self.mid_font if len(lines) > 1 else self.big_font
            rendered = [font.render(t, True, color) for t in lines]
            total_h = sum(r.get_height() for r in rendered) +
    (len(rendered)-1)*10
            start_y = HEIGHT//2 - total_h//2
            for r in rendered:
                rect = r.get_rect(center=(WIDTH//2, start_y +
    r.get_height()//2))
                self.screen.blit(r, rect)
                start_y += r.get_height() + 10


    def ask_name():
        try:
            name = input("Adını gir: ").strip()
        except EOFError:
            name = ""
        if not name: name = "Oyuncu"
        return name[:16]


    if __name__ == '__main__':
        name = ask_name()
        client = Client(name)
        client.run()
```